

Activity 13 - Document an API with rustdoc and doctests

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K5/A5

Specification

`reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs.`

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain doc comments (`///` and `//!`), `# Examples` sections, doctests and rustdoc to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_doc_comments.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on doc comments (`///` and `//!`), `# Examples` sections, doctests and rustdoc with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles writing documentation with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs.` Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Document an API with rustdoc and doctests" in `starter/src` so that the supplied tests pass. Ask the AI to draft docs from the code and requirements only; verify every example by running cargo test. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_doc_comments.rs

Run: rustc --edition 2021 --crate-type lib --crate-name docsample 01_doc_comments.rs && rustdoc --edition 2021 --test --crate-type lib --crate-name docsample -L . 01_doc_comments.rs
//! Sample library: document an API so rustdoc can render it and test it.

```
/// Days of usage kept as safety stock.
pub const SAFETY_DAYS: u32 = 2;

/// Returns the safety stock in units for a daily usage.
///
/// # Examples
/// ```
/// assert_eq!(docsample::safety_stock(12), Some(24));
/// assert_eq!(docsample::safety_stock(u32::MAX), None);
/// ```
pub fn safety_stock(daily_usage: u32) -> Option<u32> {
    daily_usage.checked_mul(SAFETY_DAYS)
}
```

Expected output:

test result: ok. 1 passed

02_errors_section.rs

Run: rustc --edition 2021 --crate-type lib --crate-name docsample2 02_errors_section.rs && rustdoc --edition 2021 --test --crate-type lib --crate-name docsample2 -L . 02_errors_section.rs
//! Sample library: documenting a function that returns Result.

```
/// Parses a whole-number quantity.
///
/// # Errors
///
/// Returns a message when `text` is not a whole number.
///
/// # Examples
/// ```
/// # fn main() -> Result<(), String> {
/// let qty = docsample2::parse_qty(" 7 ")?;
/// assert_eq!(qty, 7);
/// assert!(docsample2::parse_qty("seven").is_err());
/// # Ok(())
/// # }
/// ```
pub fn parse_qty(text: &str) -> Result<u32, String> {
    text.trim()
        .parse()
        .map_err(|_| format!("not a whole number: {text}"))
}
```

Expected output:

test result: ok. 1 passed

Learn more

Rust Book: Publishing a Crate (documentation comments):

<https://doc.rust-lang.org/book/ch14-02-publishing-to-crates-io.html>

The rustdoc Book: Documentation tests:

<https://doc.rust-lang.org/rustdoc/write-documentation/documentation-tests.html>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php